

Arrays

Rizwan Rashid



Edit with WPS Office

Opening Problem

Read one hundred numbers, compute their average, and find out how many numbers are above the average.



Introducing Arrays

- Array is a data structure that represents a collection of the same types of data
- Once an array is created, its size is fixed
- An array reference variable is used to access the elements in an array using an index



Declaring Array Variables

- `datatype[] arrayRefVar;`

Example:

```
double[] myList;
```

- `datatype arrayRefVar[];` // This style is allowed, but not preferred

Example:

```
double myList[];
```



Creating Arrays

Create an array by using the new operator and assign its reference to the variable

```
arrayRefVar = new new atype[arraySize];
```

(1) It creates an array using new
elementType[arraySize];

(2) It assigns the reference of the newly created
array to the variable arrayRefVar.

Example:

```
myList = new double[10];
```

myList[0] references the first element in the array.

myList[9] references the last element in the array.



Declaring and Creating in One Step

- `datatype[] arrayRefVar = new datatype[arraySize];`

`double[] myList = new double[10];`

- `datatype arrayRefVar[] = new datatype[arraySize];`

`double myList[] = new double[10];`

- To assign values to the elements:

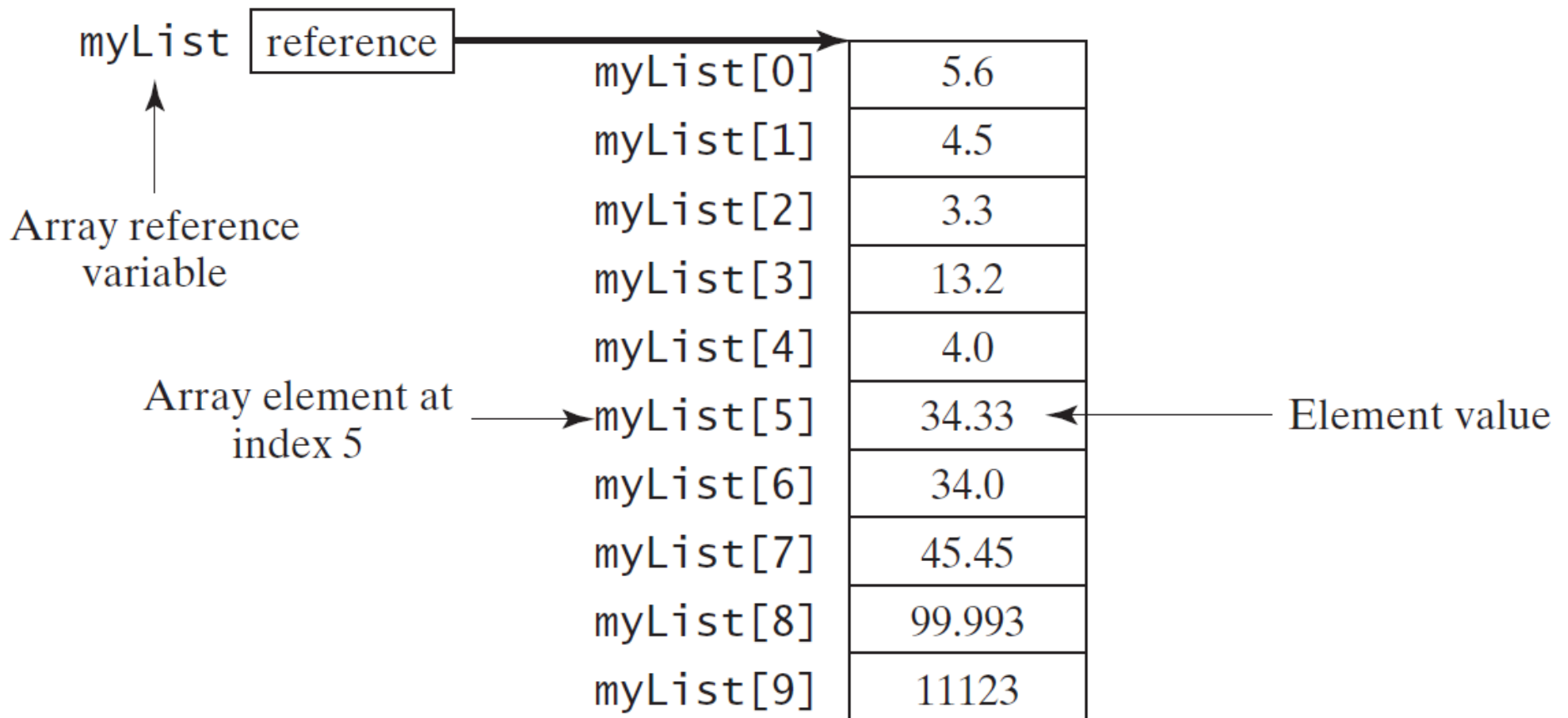
`arrayRefVar[index] = value;`

```
myList[0] = 5.6;  
myList[1] = 4.5;  
myList[2] = 3.3;  
myList[3] = 13.2;  
myList[4] = 4.0;  
myList[5] = 34.33;  
myList[6] = 34.0;  
myList[7] = 45.45;  
myList[8] = 99.993;  
myList[9] = 11123;
```



Creating Arrays

```
double[] myList = new double[10];
```



The Length of an Array

Once an array is created, its size is fixed. It cannot be changed. You can find its size using

`arrayRefVar.length`

For example,

`myList.length` returns 10



Default Values

When an array is created, its elements are assigned the default value of

0 for the numeric primitive data types,
'\u0000' for char types, and
false for boolean types.



Indexed Variables

The array elements are accessed through the index. The array indices are *0-based*, i.e., it starts from 0 to `arrayRefVar.length-1`. In the example in Figure 6.1, `myList` holds ten double values and the indices are from 0 to 9.

Each element in the array is represented using the following syntax, known as an *indexed variable*:

```
arrayRefVar[index];
```



Using Indexed Variables

After an array is created, an indexed variable can be used in the same way as a regular variable. For example, the following code adds the value in `myList[0]` and `myList[1]` to `myList[2]`.

```
myList[2] = myList[0] + myList[1];
```



Declaring, creating, initializing Using the Shorthand Notation

```
double[] myList = {1.9, 2.9, 3.4, 3.5};
```

- This shorthand syntax must be in one statement.

This shorthand notation is equivalent to the following statements:

```
double[] myList = new double[4];
```

```
myList[0] = 1.9;
```

```
myList[1] = 2.9;
```

```
myList[2] = 3.4;
```

```
myList[3] = 3.5;
```



Processing Arrays

1. (Initializing arrays with input values)
2. (Initializing arrays with random values)
3. (Printing arrays)
4. (Summing all elements)
5. (Finding the largest element)
6. (Finding the smallest index of the largest element)
7. (*Random shuffling*)
8. (*Shifting elements*)



Initializing arrays with input values

```
java.util.Scanner input = new  
    java.util.Scanner(System.in);  
System.out.print("Enter " + myList.length + " values: ");  
for (int i = 0; i < myList.length; i++)  
    myList[i] = input.nextDouble();
```



Initializing arrays with random values

```
for (int i = 0; i < myList.length; i++) {  
    myList[i] = Math.random() * 100;  
}
```



Printing arrays

```
for (int i = 0; i < myList.length; i++) {  
    System.out.print(myList[i] + " ");  
}
```



Summing all elements

```
double total = 0;  
for (int i = 0; i < myList.length; i++) {  
    total += myList[i];  
}
```



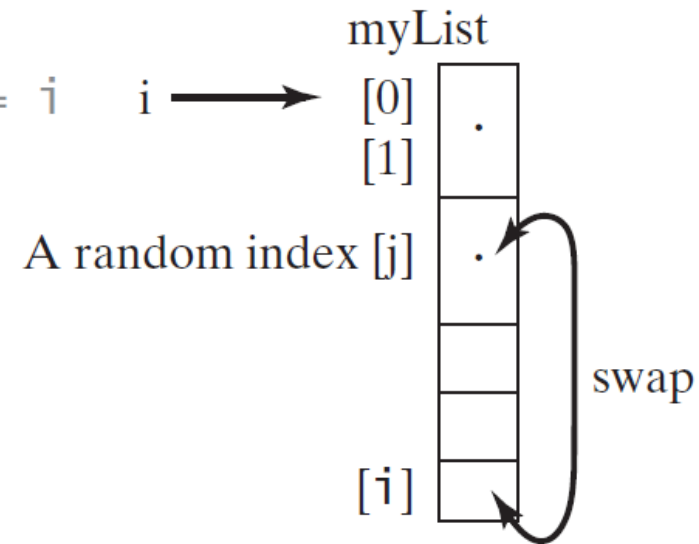
Finding the largest element

```
double max = myList[0];  
for (int i = 1; i < myList.length; i++) {  
    if (myList[i] > max)  
        max = myList[i];  
}
```



Random shuffling

```
for (int i = myList.length - 1; i > 0; i--) {  
    // Generate an index j randomly with 0 <= j <= i  
    int j = (int)(Math.random()  
        * (i + 1));  
  
    // Swap myList[i] with myList[j]  
    double temp = myList[i];  
    myList[i] = myList[j];  
    myList[j] = temp;  
}
```



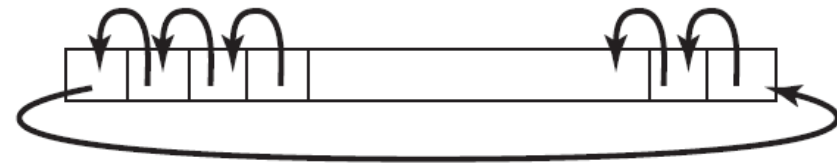
Shifting Elements

```
double temp = myList[0]; // Retain the first element
```

```
// Shift elements left  
for (int i = 1; i < myList.length; i++) {  
    myList[i - 1] = myList[i];  
}
```

```
// Move the first element to fill in the last position  
myList[myList.length - 1] = temp;
```

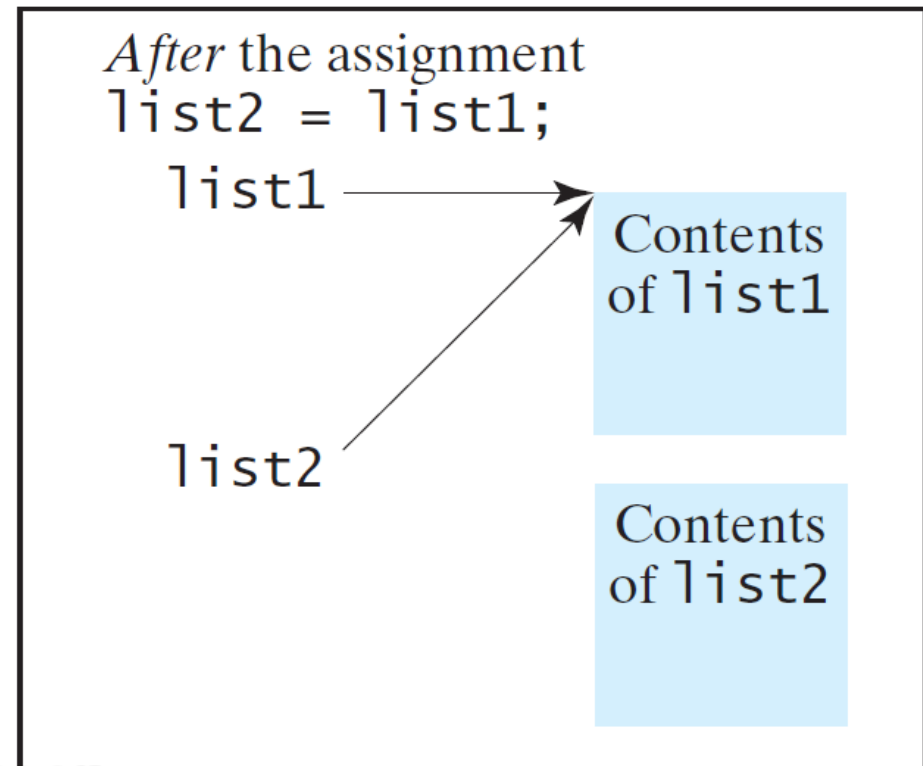
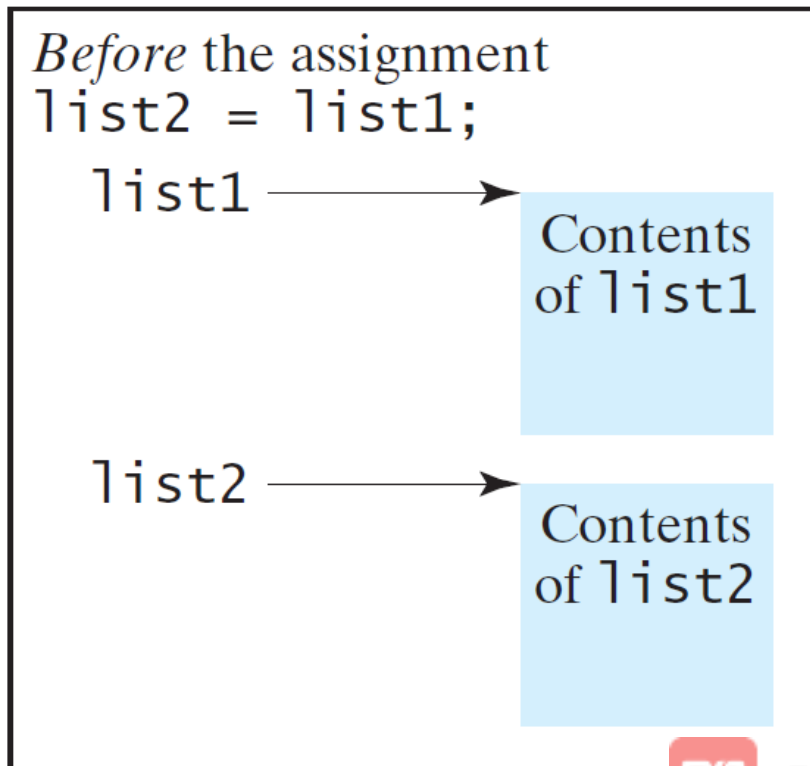
myList



Copying Arrays

Often, in a program, you need to duplicate an array or a part of an array. In such cases you could attempt to use the assignment statement (=), as follows:

```
list2 = list1;
```



Copying Arrays

To copy the contents of one array into another, you have to copy the array's individual elements into the other array

Using a loop:

```
int[] sourceArray = {2, 3, 1, 5, 10};  
int[] targetArray = new int[sourceArray.length];  
  
for (int i = 0; i < sourceArray.length; i++)  
    targetArray[i] = sourceArray[i];
```



The arraycopy Utility

```
arraycopy(sourceArray, src_pos, targetArray, tar_pos, length);
```

- **srcPos** and **tarPos** indicate the starting positions in **sourceArray** and **targetArray**, respectively
- **length** number of elements copied from **sourceArray** to **targetArray**

Example:

```
System.arraycopy(sourceArray, 0, targetArray, 0, sourceArray.length);
```



Passing Arrays to Methods

When passing an array to a method, the reference of the array is passed to the method.

```
public static void printArray(int[] array) {  
    for (int i = 0; i < array.length; i++) {  
        System.out.print(array[i] + " ");  
    }  
}
```

Invoke the method

```
int[] list = {3, 1, 2, 6, 4, 2};  
printArray(list);
```

Anonymous Array

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```



Edit with WPS Office

Pass By Value

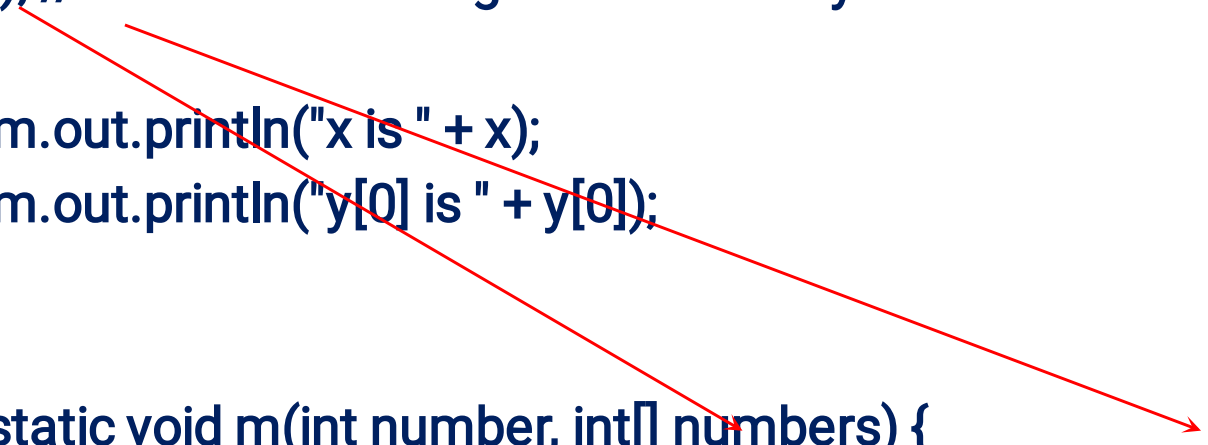
Java uses *pass by value* to pass arguments to a method. There are important differences between passing a value of variables of primitive data types and passing arrays.

- For a parameter of a primitive type value, the actual value is passed. Changing the value of the local parameter inside the method does not affect the value of the variable outside the method.
- For a parameter of an array type, the value of the parameter contains a reference to an array; this reference is passed to the method. Any changes to the array that occur inside the method body will affect the original array that was passed as the argument.



Simple Example

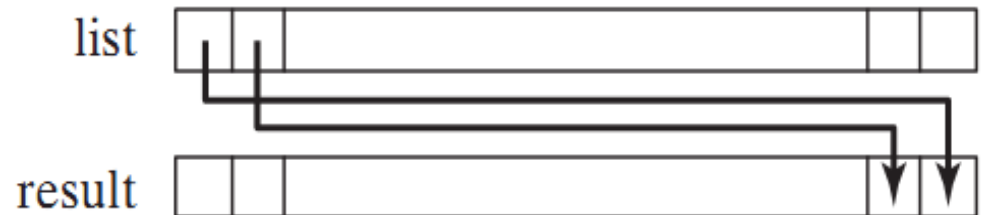
```
public class Test {  
    public static void main(String[] args) {  
        int x = 1; // x represents an int value  
        int[] y = new int[10]; // y represents an array of int values  
  
        m(x, y); // Invoke m with arguments x and y  
  
        System.out.println("x is " + x);  
        System.out.println("y[0] is " + y[0]);  
    }  
  
    public static void m(int number, int[] numbers) {  
        number = 1001; // Assign a new value to number  
        numbers[0] = 5555; // Assign a new value to numbers[0]  
    }  
}
```



Returning an Array from a Method

When a method returns an array, the reference of the array is returned

```
1 public static int[] reverse(int[] list) {  
2     int[] result = new int[list.length];  
3  
4     for (int i = 0, j = result.length - 1;  
5         i < list.length; i++, j--) {  
6         result[j] = list[i];  
7     }  
8  
9     return result;  
10 }
```



Variable-Length Argument Lists

A variable number of arguments of the same type can be passed to a method and treated as an array.

- You can pass a variable number of arguments of the same type to a method
- Syntax: **typeName... parameterName**
- Java treats a variable-length parameter as an array
- Only one variable-length parameter may be specified in a method, and this parameter must be the last parameter
- Any regular parameters must come before it.

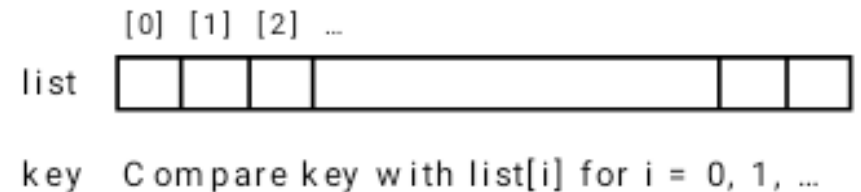
public static void varLengthMethod(double a,int...n)



Searching Arrays

Searching is the process of looking for a specific element in an array; for example, discovering whether a certain score is included in a list of scores. Searching is a common task in computer programming. There are many algorithms and data structures devoted to searching. In this section, two commonly used approaches are discussed, *linear search* and *binary search*.

```
public class LinearSearch {  
    /** The method for finding a key in the list */  
    public static int linearSearch(int[] list, int key) {  
        for (int i = 0; i < list.length; i++)  
            if (key == list[i])  
                return i;  
        return -1;  
    }  
}
```



Linear Search

The linear search approach compares the key element, key, *sequentially* with each element in the array list. The method continues to do so until the key matches an element in the list or the list is exhausted without a match being found. If a match is made, the linear search returns the index of the element in the array that matches the key. If no match is found, the search returns -1.



Linear Search Animation

Key	List
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8
3	6 4 1 9 7 3 2 8



From Idea to Solution

```
/** The method for finding a key in the list */  
public static int linearSearch(int[] list, int key) {  
    for (int i = 0; i < list.length; i++)  
        if (key == list[i])  
            return i;  
    return -1;  
}
```

Trace the method

```
int[] list = {1, 4, 4, 2, 5, -3, 6, 2};  
int i = linearSearch(list, 4); // returns 1  
int j = linearSearch(list, -4); // returns -1  
int k = linearSearch(list, -3); // returns 5
```



Binary Search

For binary search to work, the elements in the array must already be ordered. Without loss of generality, assume that the array is in ascending order.

e.g., 2 4 7 10 11 45 50 59 60 66 69 70 79

The binary search first compares the key with the element in the middle of the array.



Binary Search, cont.

Consider the following three cases:

- If the key is less than the middle element, you only need to search the key in the first half of the array.
- If the key is equal to the middle element, the search ends with a match.
- If the key is greater than the middle element, you only need to search the key in the second half of the array.



Binary Search

Key

List

8

1	2	3	4	6	7	8	9
---	---	---	---	---	---	---	---

8

1	2	3	4	6	7	8	9
---	---	---	---	---	---	---	---

8

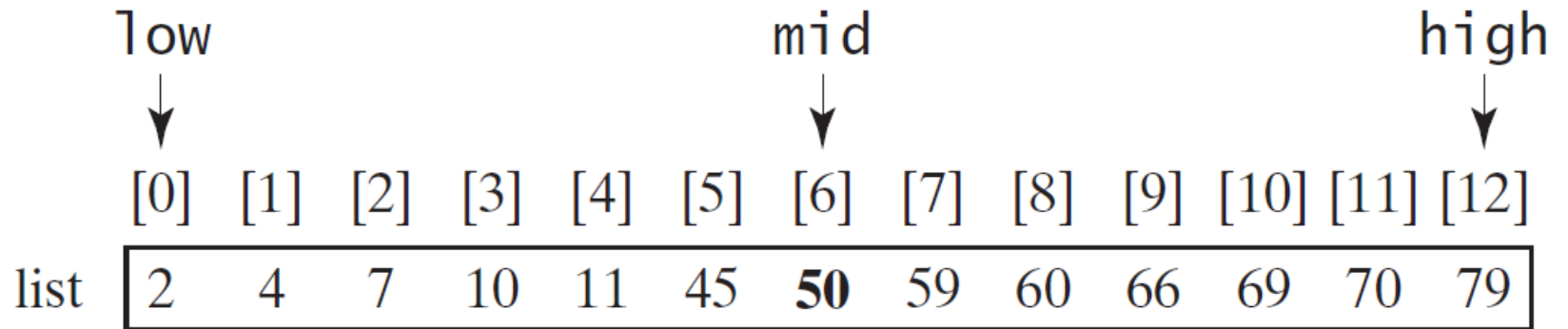
1	2	3	4	6	7	8	9
---	---	---	---	---	---	---	---



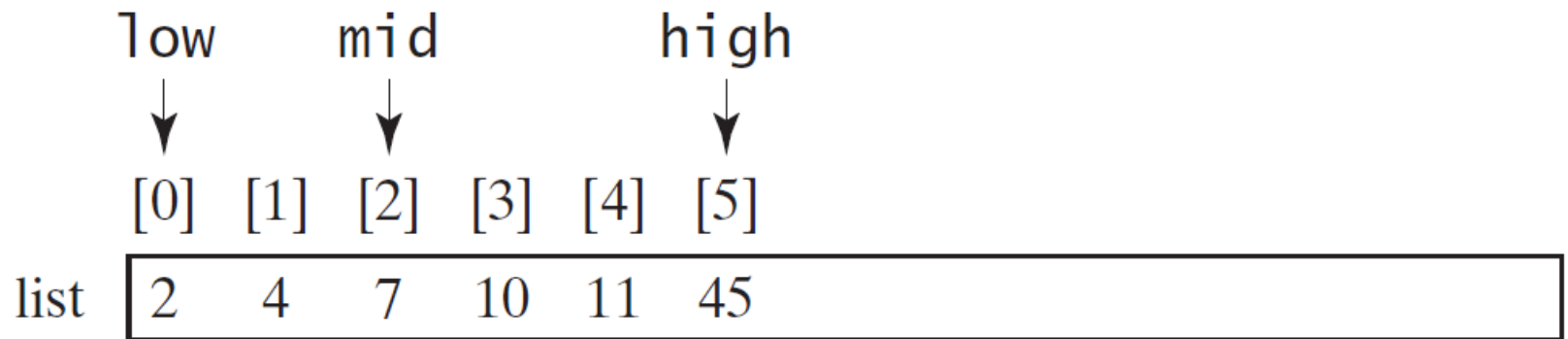
Binary Search, cont.

key is 11

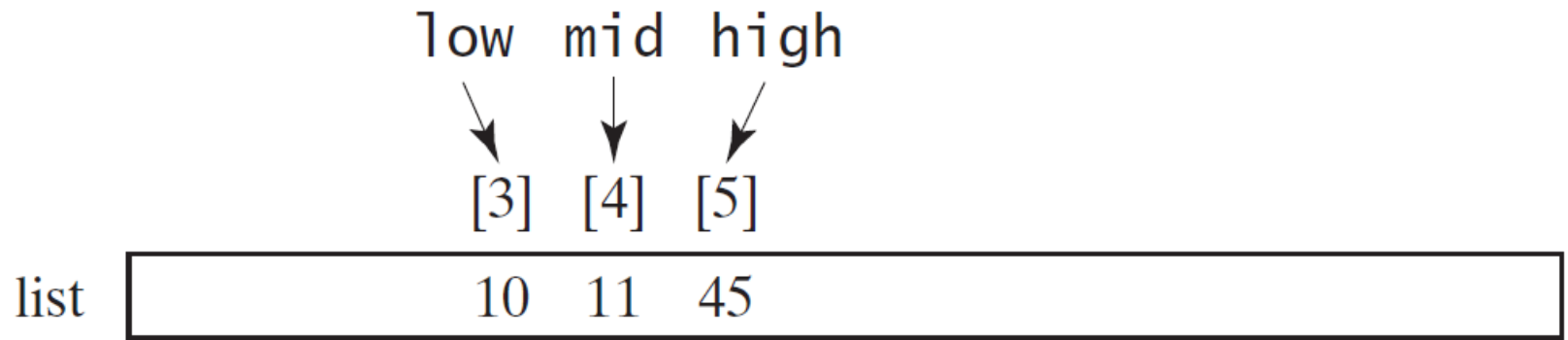
key < 50



key > 7



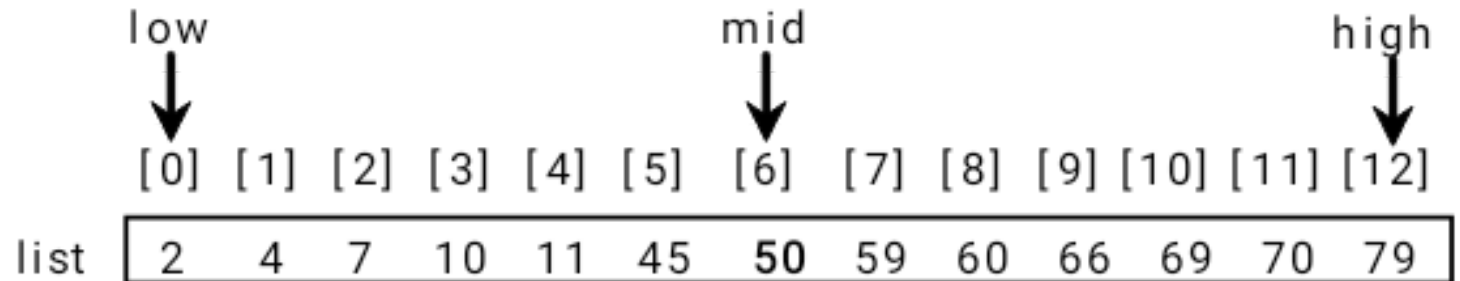
key == 11



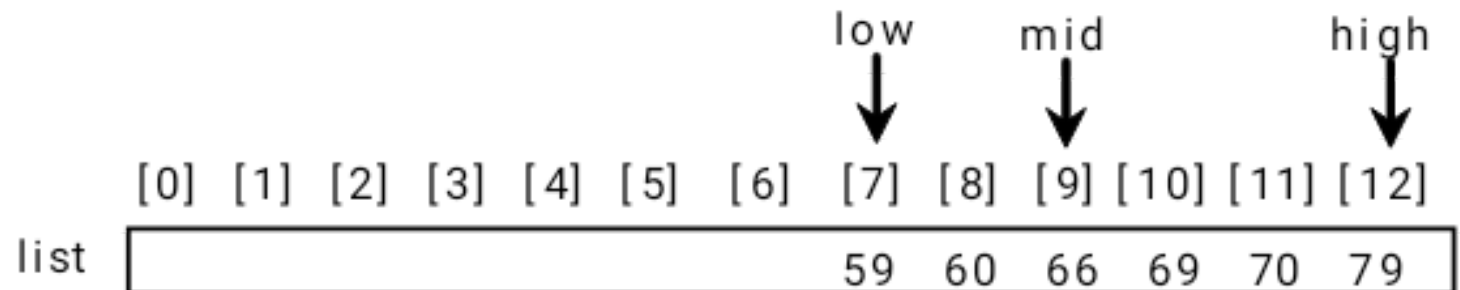
Binary Search, cont.

key is 54

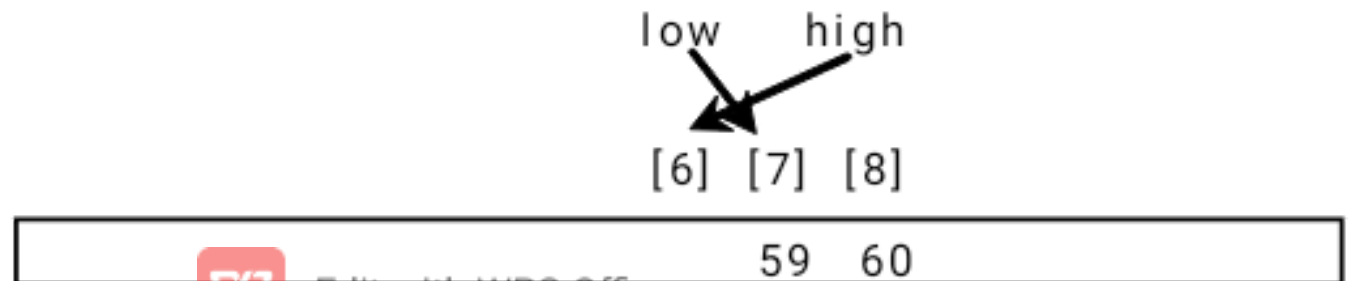
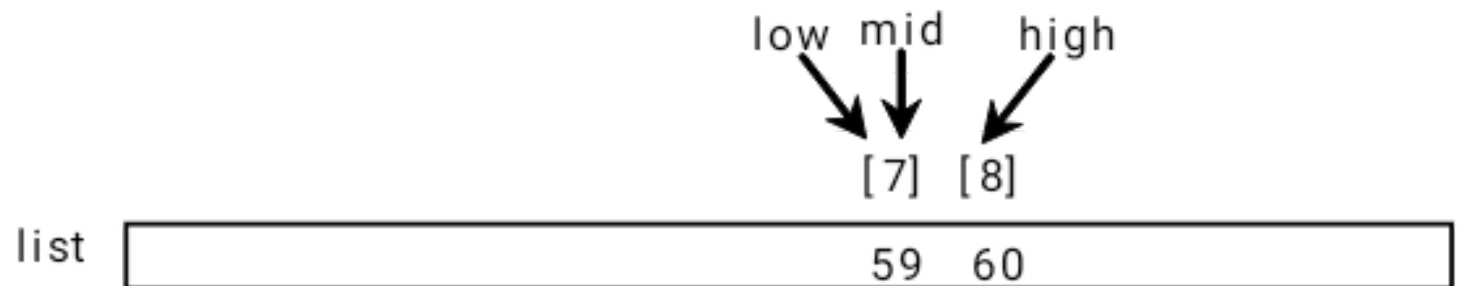
key > 50



key < 66



key < 59



Binary Search, cont.

The `binarySearch` method returns the index of the element in the list that matches the search key if it is contained in the list. Otherwise, it returns

-insertion point - 1.

The insertion point is the point at which the key would be inserted into the list.



From Idea to Solution

```
/** Use binary search to find the key in the list */  
public static int binarySearch(int[] list, int key) {  
    int low = 0;  
    int high = list.length - 1;  
  
    while (high >= low) {  
        int mid = (low + high) / 2;  
        if (key < list[mid])  
            high = mid - 1;  
        else if (key == list[mid])  
            return mid;  
        else  
            low = mid + 1;  
    }  
  
    return -1 - low;  
}
```



The Arrays.binarySearch Method

Since binary search is frequently used in programming, Java provides several overloaded binarySearch methods for searching a key in an array of int, double, char, short, long, and float in the java.util.Arrays class. For example, the following code searches the keys in an array of numbers and an array of characters.

```
int[] list = {2, 4, 7, 10, 11, 45, 50, 59, 60, 66, 69, 70, 79};  
System.out.println("Index is " +  
    java.util.Arrays.binarySearch(list, 11));
```

Return is 4

```
char[] chars = {'a', 'c', 'g', 'x', 'y', 'z'};  
System.out.println("Index is " +  
    java.util.Arrays.binarySearch(chars, 't'));
```

Return is -4 (insertion point is 3, so return is -3-1)

For the binarySearch method to work, the array must be pre-sorted in increasing order.



Sorting Arrays

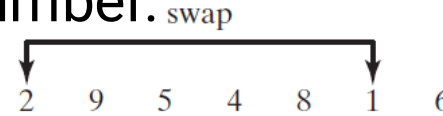
Sorting, like searching, is also a common task in computer programming. Many different algorithms have been developed for sorting. This section introduces a simple, intuitive sorting algorithm: *selection sort*.



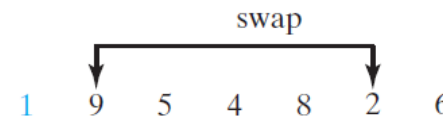
Selection Sort

Selection sort finds the smallest number in the list and places it first. It then finds the smallest number remaining and places it second, and so on until the list contains only a single number.

Select 1 (the smallest) and swap it with 2 (the first) in the list.

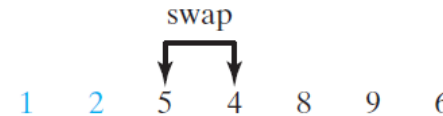


The number 1 is now in the correct position and thus no longer needs to be considered.



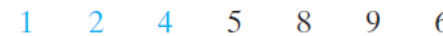
Select 2 (the smallest) and swap it with 9 (the first) in the remaining list.

The number 2 is now in the correct position and thus no longer needs to be considered.



Select 4 (the smallest) and swap it with 5 (the first) in the remaining list.

The number 4 is now in the correct position and thus no longer needs to be considered.



5 is the smallest and in the right position. No swap is necessary.

The number 5 is now in the correct position and thus no longer needs to be considered.



Select 6 (the smallest) and swap it with 8 (the first) in the remaining list.

The number 6 is now in the correct position and thus no longer needs to be considered.



Select 8 (the smallest) and swap it with 9 (the first) in the remaining list.

The number 8 is now in the correct position and thus no longer needs to be considered.



Since there is only one element remaining in the list, the sort is completed.



Wrap it in a Method

```
/** The method for sorting the numbers */  
public static void selectionSort(double[] list) {  
    for (int i = 0; i < list.length; i++) {  
        // Find the minimum in the list[i..list.length-1]  
        double currentMin = list[i];  
        int currentMinIndex = i;  
        for (int j = i + 1; j < list.length; j++) {  
            if (currentMin > list[j]) {  
                currentMin = list[j];  
                currentMinIndex = j;  
            }  
        }  
  
        // Swap list[i] with list[currentMinIndex] if necessary;  
        if (currentMinIndex != i) {  
            list[currentMinIndex] = list[i];  
            list[i] = currentMin;  
        }  
    }  
}
```

Invoke it
selectionSort(yourList)



The Arrays.sort Method

Since sorting is frequently used in programming, Java provides several overloaded sort methods for sorting an array of int, double, char, short, long, and float in the `java.util.Arrays` class. For example, the following code sorts an array of numbers and an array of characters.

```
double[] numbers = {6.0, 4.4, 1.9, 2.9, 3.4, 3.5};  
java.util.Arrays.sort(numbers);
```

```
char[] chars = {'a', 'A', '4', 'F', 'D', 'P'};  
java.util.Arrays.sort(chars);
```



The Arrays.toString(list) Method

The Arrays.toString(list) method can be used to return a string representation for the list.



Main Method Is Just a Regular Method

You can call a regular method by passing actual parameters. Can you pass arguments to main? Of course, yes. For example, the main method in class B is invoked by a method in A, as shown below:

```
public class A {  
    public static void main(String[] args) {  
        String[] strings = {"New York",  
                             "Boston", "Atlanta"};  
        B.main(strings);  
    }  
}
```

```
class B {  
    public static void main(String[] args) {  
        for (int i = 0; i < args.length; i++)  
            System.out.println(args[i]);  
    }  
}
```



Command-Line Parameters

```
class TestMain {  
    public static void main(String[] args) {  
        ...  
    }  
}
```

```
java TestMain arg0 arg1 arg2 ... argn
```



Processing Command-Line Parameters

In the main method, get the arguments from `args[0]`, `args[1]`, ..., `args[n]`, which corresponds to `arg0`, `arg1`, ..., `argn` in the command line.

